Never Twice the Same Mistake: Counterexample-Guided Program Repair with a Location-Indexed Memory of Refuted Attempts

Anonymous Author(s)
Submitted for double-anonymous review

***Abstract*—Large language model repair agents iterate: propose a patch, run the tests, read the failure, try again. Recent measurement shows how expensive the running is and how little of it pays—agents average 8.8 test executions per SWE-bench task and "apply execution indiscriminately." Classical repair cuts that work either by relating candidate patches to one another, which an unenumerable candidate space rules out, or by ordering the tests so a doomed candidate dies on the first, which nothing rules out.**

We carry the second mechanism into the LLM loop and change what it orders from. CEGMEM retains counterexamples and indexes them by edit location—the half of a failure label a diff supplies without executing anything—then re-executes a stored one against each new candidate, resolving the round as soon as an input already in hand still breaks it. In the synthesis loop this idea comes from such replay would be vacuous, since a constrained synthesizer cannot emit a candidate its own counterexamples refute; an LLM repeats itself in one proposal out of four. We prove guard soundness against a finite oracle, a $K{+}1$ round bound and strict redundancy reduction, and show the index is *not* asymptotically cheaper than a flat scan.

We then run the loop on 99 ConDefects faults with a local `qwen2.5-coder:7b` proposer, five arms and 1,485 pre-specified main-grid cells, every arm paired by common random numbers so round 1 is the identical draw in all of them. Memory cuts oracle calls per episode $\times 5.2$ and—counting guard replays as the executions they are—total program executions $\times 2.2$, at unchanged repair rate; at a matched budget of two oracle calls it lifts repair from 31.9% to 55.8% (task-clustered 95% CI on the gain $[18.4, 29.5]$ pp). One result cuts against the architecture as proposed and we report it as primary: prompt-side steering is null on every outcome metric at $+128\%$ prompt tokens, and no band survives an interaction test. The guard alone, not the full typed agent, is what we recommend.

***Index Terms*—automated program repair, large language models, agent memory, counterexample-guided repair, test execution cost, empirical software engineering**

I. INTRODUCTION

Automated program repair has moved from search- and template-based systems to agents built on large language models that repair real projects by interacting with an environment: localize a fault, propose an edit, run the test suite, read the failure, revise [1]–[5]. That loop is counterexample-guided repair by another name—each failing test is a counterexample that ought to constrain the next proposal, exactly as a failing input drives the synthesis loop of SKETCH [6].

a) The cost of that loop is now measured.: Analyzing $7{,}745$ SWE-bench agent traces and $3{,}000$ end-to-end repair runs, Lin et al. [7] report that agents average 8.8 test executions per task, that the benefit concentrates on a minority of instances, and that restricting execution entirely costs only 1.25 percentage points of resolve rate; their conclusion is that "current agents apply execution indiscriminately, paying their cost on instances where it provides little benefit." Agent-authored tests likewise "mimic a familiar software-development practice while consuming interaction budget" [8], and counting iteration honestly makes self-repair's gains "often modest" [9].

Three responses exist. Systems work makes each validation cheaper [10], [11]; scaling work buys accuracy with money, most explicitly CodeMonkeys at ≈\$2,300 for 57.4% of SWE-bench Verified [12]; and classical generate-and-validate APR cuts the work itself, either by relating candidates to each other [13], [14], which needs an enumerable candidate space an LLM does not supply, or by ordering the tests so a doomed candidate dies on the first [15]–[17], which needs nothing of the candidate space at all. The second mechanism transfers. Our question is what it should order from when the generator is a language model.

b) A refutation you already hold is such a criterion.: If memory contains a counterexample x that a candidate p still fails, p is refuted and validating it further buys nothing. In the synthesis loop this idea comes from, the criterion could never fire—a constrained synthesizer cannot emit a candidate that its accumulated counterexamples refute—but an LLM proposer is under no such obligation and repeats itself in one proposal out of four (Section II-C). The obstacle is that today's agents keep a flat conversational transcript [5], [18], [19], which neither *retrieves*—it dilutes as the dialogue grows—nor *generalizes*: it records that patch p_3 failed test t, not that every patch sharing p_3's defect will fail the same way.

c) Our approach: keep the evidence, index it by where the patch edits.: When a proposal p is refuted by counterexample x, CEGMEM stores (p, x, τ) where $\tau = \theta(p, x)$ pairs p's edit location with the property the execution violated. The halves are available at different times, and the mechanism rests on that: $\lambda(p)$ is a diff against the buggy source and needs no execution,

while the violated property exists only after a failing run. So the store is *indexed by* λ and can be consulted for a candidate nobody has run.

That buys two operations. The agent can *guard*: re-execute a stored counterexample against the new candidate and, if it still fails, resolve the round without calling the oracle—a decision that is verified rather than predicted, since the candidate is blocked only after failing an input the oracle itself produced. Inlined into the oracle this is a prioritization discipline that stops on the first failing case, and we claim no more than that; what is new is that the order comes from counterexamples the loop discovered itself and kept across candidates. And it can *steer*: remove already-refuted classes from the proposer's support. Our study finds the first carries essentially all of the value and the second none, which is not what we set out to show.

d) This paper.: We give CEGMEM a formal core (Section IV) and run it on real faults rather than the simulation the design was first argued from: 99 ConDefects faults, a local `qwen2.5-coder:7b` proposer at $T = 1.0$, a differential oracle over the shipped pool, five arms and four sweeps, 1,485 pre-specified main-grid cells and 3,654 episodes. Two commitments make the comparison tight: every arm is paired by *common random numbers* [20], so round 1 is the byte-identical draw in every arm and later divergence is the treatment's own doing; and the primary metric is pre-specified as repair rate at a matched *oracle-call* budget, not total calls, which an arm without a guard loses by construction.

Memory cuts oracle calls $\times 5.2$ and, counting every guard replay as the execution it is, total program executions $\times 2.2$, at unchanged repair rate; at a matched two-call budget it lifts repair from 31.9% to 55.8%. Two results cut against the architecture as proposed and we report them as primary: prompt-side steering is null on every outcome metric while costing $+128\%$ prompt tokens and making the full agent the slowest of five arms, with no band surviving a formal interaction test. The configuration we recommend is consequently the guard alone.

e) Contributions.: *(i)* An architecture separating verification-side guarding from prompt-side steering into independently analyzable operators (Section III). *(ii)* A formal core stated against a *finite* oracle: guard soundness, a $K+1$ round bound, strict redundancy reduction, and an explicit account of what indexing does and does not buy (Section IV). *(iii)* A controlled execution of that loop on real faults under common random numbers, with pre-specified metrics, falsifiers and a random-partition control (Sections VI and VII). *(iv)* A negative result reported as a result: the prompt-side half does not work at this scale, which relocates the value—and the recommendation—onto the guard (Section VIII).

II. Background and Related Work

Deciding that one candidate patch needs less validation work than the next is not new, and Table I sorts the ways it has been done by what the decision is computed from and what that demands of the candidate space. We put CEGMEM in the same row as a mechanism a decade older than LLM repair, and then say what is left over.

A. Removing validations: candidate equivalence

One family relates candidates *to each other* and validates one representative per class. AE quotients the patch space under an approximate program-equivalence relation: on the 45 defects it and GenProg both repair, from a 105-defect benchmark, it needs 186 test-suite evaluations against $3{,}252$ [13]. Test-equivalence analysis partitions candidates by the values a modified expression evaluates to during a test, so one execution decides an entire class [14], and ExpressAPR detects that equivalence *online*, reporting $\approx 137\times$ over plain validation on Defects4J [11]. UniAPR, by contrast, purely accelerates—it redefines bytecode inside a warm JVM and still runs every test [10]. The equivalence mechanisms give a stronger guarantee than ours, and they need what an LLM does not supply: an enumerable candidate space and an edit shape the analysis can model. That is why they do not transfer, and it is the only reason.

B. Reordering validations, and where our guard belongs

The other family validates every candidate and changes the order in which evidence against it is sought. Fault-recorded prioritization runs the tests that killed earlier candidates first [15]; modification-point–aware prioritization keeps one ordered suite per edit location [16]; AE's TestStrat—in the same paper as its quotient space—takes the test likeliest to fail and stops there [13]; and a study of 2,532,915 patches from 12 repair systems on 395 Defects4J bugs quantifies what selection and prioritization save [17].

Our guard belongs here, and we state the identity rather than argue with it. Inline the guard into the oracle—stored counterexamples first, stop at the first failure—and the executions performed are the same ones, in the same order, at the same cost. Nothing is *removed*; a validation is made to terminate on its first case. We therefore claim an ordering effect and report it as one: in TYPED a round the guard resolves costs 1.13 program executions where a round reaching the oracle costs 18.9, and memory turns roughly 7.6 oracle calls per episode into roughly 7.4 early-terminating ones (Section VII-A). What is *not* shared with prior work is the key and the evidence. Those tables rank the project's own tests by a failure history over candidates; ours holds inputs discovered inside the episode by candidates the oracle already refuted, re-executed against a candidate nobody has run, indexed by a location a diff supplies. The per-modification-point table of [16] is the nearest neighbour, and the baseline this paper most needs (Section IX).

C. Counterexample-guided repair, and why replay is not vacuous

That a counterexample should constrain the next candidate is the engine of the loop SKETCH introduced [6], and the loop has been run with an LLM in the synthesizer's place over $1{,}431$ student C programs [23]. There, retaining counterexamples and

TABLE I

WHERE CEGMEM SITS. THE COLUMNS ARE FACTUAL PROPERTIES OF EACH MECHANISM, NOT A SCORING OF IT. CEGMEM SHARES THE LAST COLUMN'S VALUE WITH A LINE A DECADE OLDER THAN LLM REPAIR; WHAT IS NEW IS THE KEY IT ORDERS BY, THE EVIDENCE IT ORDERS FROM, AND THAT IT NEEDS NOTHING OF THE CANDIDATE SPACE—EXACTLY WHAT THE TWO *removing* ROWS REQUIRE AND AN LLM CANNOT SUPPLY.

Line of work	**Representatives**	**The decision is computed from**	**Requires of the candidate space**	**Effect on validation work**
Candidate equivalence	AE [13]; f1x [14]	candidates related to *each other*: syntactically, or by the values a modified expression takes	enumerable; an edit shape the analysis models	removes
Validation engineering	UniAPR [10]; ExpressAPR [11]	a warm JVM; online patch equivalence, schemata, virtualization	mutation-shaped patches, for the equivalence part	accelerates; ExpressAPR also removes
Test prioritization	fault-recorded [15]; modification-point aware [16]; AE's TestStrat [13]; RTS study [17]	which of the project's own tests killed earlier candidates, keyed on nothing or on the edit location	nothing	reorders, stops early
LLM repair agents	RepairAgent [1]; SWE-agent [3]; Agentless [4]; ChatRepair [5]; ExpeRepair [21]; [22]	a flat transcript, or demonstrations and insights, retrieved into the prompt	nothing	none
Counterexample-guided repair	SKETCH [6]; LLM CEGIS [23]	counterexamples, as prompt content for the next attempt	a complete synthesizer (SKETCH); nothing (LLM)	none
CEGMEM	this paper	counterexamples *retained across candidates* and re-executed, indexed by edit location λ	nothing	reorders, stops early

replaying them would be *vacuous*: a constrained synthesizer cannot emit a candidate one of them refutes, so a guard would never fire. Losing that constraint is what creates the opportunity, and an LLM takes it often. In GUARD-ONLY, 24.2% of proposals are byte-identical to an earlier one in the same episode and 30.2% of the rounds the guard resolves are such a repeat; of those resolutions 12.8% re-propose a patch the store already holds, and the remaining 87.2% are patches the store never held, refuted by a stored input anyway (the supplement). Neither case can arise in SKETCH. Orvalho et al. use the counterexample as prompt content for the next attempt—our STEER-ONLY arm, the half we find null; we add the second use of the same evidence.

D. Memory in LLM repair agents

LLMs repair code by conversing: ChatRepair iterates on test failures [5], agentic systems localize, edit and validate over many turns [1]–[3], "agentless" pipelines keep a fixed propose–validate workflow [4], and general frameworks add verbal self-reflection [18], [19]. In all of them the record of past attempts is a flat transcript and the cost model is dollars or tokens. The closest memory-augmented system, ExpeRepair [21], retrieves demonstrations and reflective insights to condition generation but never validation; nothing in that index recognizes that a new candidate is refuted by an input already in hand. Vo et al. [22] type failures for text-to-SQL repair, conditioning *retrieval for the prompt* where we condition *the order of validation*, and report their typed variant "is not reliably separated from untyped dynamic retrieval"—the phenomenon we measure directly. The guard is a property of memory, not of a controller, so it is orthogonal to every agent above. Finally, since a patch can pass the visible tests and still be wrong [24] and LLM-era benchmarks must worry about leakage, we use ConDefects [25] rather than Defects4J [26] or SWE-bench [27], and treat acceptance as a claim about G_k audited against G_{pool} (Section VII-D).

III. THE CEGMEM ARCHITECTURE

A. Problem setup

A repair task is a tuple $(\mathcal{P}, \varphi, \mathcal{O}_k)$: a candidate space of patches, a specification, and an oracle. A patch is *semantically correct* iff it satisfies φ on every input in a domain $\mathcal{X}$, and we write $G^\star$ for that set, assumed non-empty. No implementable oracle decides membership in $G^\star$. What we have is a *counterexample oracle at depth* k: a call draws a check set X_k of $\min(k, |\text{pool}|)$ inputs from the fault's shipped pool and runs them until one separates candidate from reference,

$$\mathcal{O}_k(p) = \begin{cases} x \in X_k \text{ with } \neg\varphi(p,x) & \text{if a checked input fails,} \\ \textsf{accept} & \text{otherwise.} \end{cases} \tag{1}$$

Write G_k for what one such call certifies—the patches passing every $x \in X_k$—and G_{pool} for those passing the *entire* pool. Two properties of X_k are load-bearing. It is drawn *per call*, so G_k is a family indexed by the draw; but the draw is degenerate whenever $k \geq |\text{pool}|$, where X_k is the pool and $G_k = G_{\text{pool}}$—the case for 97 of our 99 faults at the $k = 100$ every headline number uses. The three sets nest, $G^\star \subseteq G_{\text{pool}} \subseteq G_k$, so acceptance is the weakest of them and $G_k \setminus G^\star$ is *false acceptance*; Section VII-D measures that gap and never assumes it away. Three budget quantities also stay distinct: the proposal budget B, the oracle budget b against which repair rate is reported, and the oracle depth k.

B. Failure labels, and the key available before execution

A refuted candidate p with counterexample x receives a label

$$\theta(p,x) = \big(\lambda(p),\ \text{property violated at } x\big) \in \mathcal{T}, \tag{2}$$

where $\lambda(p)$ is the patch's *edit location*, obtained by diffing the candidate against the buggy source. The two components differ in when they are available, and that is load-bearing: $\lambda(p)$ needs no execution, while the violated property is only defined *after* a failing run and so does not exist for a fresh candidate.

The guard therefore cannot index on θ. It indexes on λ, and memory's buckets are keyed on it: $\mathcal{M}[\ell] = \{(p', x', \tau') \in \mathcal{M} : \lambda(p') = \ell\}$. The full label θ is what steering excludes—it names classes already *observed*, for which the property component exists—and what the coherence sweep perturbs. Nothing here requires predicting a violated property before running anything.

Definition 1 (Informativeness and coherence)**.** The oracle is ρ-*informative* if a counterexample refuting a candidate of class τ refutes every candidate of class τ with probability at least ρ. Memory is c-*coherent* if it attributes a candidate or counterexample to its true class with probability at least c. The ideal regime is $\rho = c = 1$.

C. Typed memory: guard and steer

CEGMEM maintains a memory $\mathcal{M}$ of triples (p, x, τ) and the induced set of eliminated classes E. It exposes two operations, kept distinct because Section VII shows they do different jobs and that only one works.

a) Guard.: Before paying the oracle, the guard rejects a candidate that a stored counterexample still refutes:

$$\mathrm{Guard}_{\mathcal{M}}(p) = \mathbb{1}[\exists (p', x', \tau') \in \mathcal{M} : x' \text{ refutes } p]\,. \quad (3)$$

Here x' *refutes* p under the oracle's own predicate—p on x' disagrees with the reference, crashes, or times out—so a block and a refutation are the same event, evaluated by different callers. An *indexed* guard evaluates Eq. (3) against $\mathcal{M}[\lambda(p)]$ first and then the rest of memory—the fallback belongs to the analyzed algorithm and to the shipped one, not to one of them—while an *untyped* guard walks memory in insertion order. Both short-circuit at the first entry that still refutes and both consult all of memory when none does, so their worst cases coincide: the index changes the *order* of consultation, not its bound (the index-cost proposition). Our implementation additionally deduplicates stored counterexamples by input and memoizes verdicts within a round, reducing executions at unchanged consultations.

Two properties of Eq. (3) are easy to lose. It is a *verified* predicate, not a match heuristic: a block happens only when a stored counterexample is re-executed and still fails, so widening the search can never cost soundness (Theorem 2). And the index must say where to look *first*, not where to *stop*: λ records where a candidate edits while refutation is decided by which input it gets wrong, so a bucket-only guard is an unsound stopping rule. Section V reports what that cost us.

b) Steer.: Memory also reshapes the proposer. Let w_s be its base mass on class s, with $w_\top = \pi$ for the correct class and $w_\tau = q_\tau$ otherwise. The steered proposer renormalizes onto the surviving support:

$$\mathbb{P}_{\mathrm{steer}}[s] = \frac{w_s \cdot \mathbb{1}[s \notin E]}{\pi + \sum_{\tau \notin E} q_\tau}, \qquad s \in \{\top\} \cup \mathcal{T}. \quad (4)$$

This is what a flat transcript cannot do—excluding a class requires having one—and it idealizes an operation an LLM can only be *asked* to perform: an exclusion instruction plus class-matched evidence in the prompt. Whether the request is honored is empirical; Section VII-C answers no for a 7B proposer, while the same renormalization applied on the guard side, by not charging blocked rounds, does deliver (Section VII-C).

D. The repair loop

Each round draws a steered proposal, guards it against memory, and calls the oracle only if it survives; an accepted patch is returned and a refuted one stored with its counterexample and label, eliminating that class. The arms in Section VII switch the two operations independently (the supplement gives the loop in full). CEGMEM adds no planner and no controller—the only new state is the memory—which is what keeps Section IV's guarantees properties of *memory*, and so independent of the proposer.

IV. Formal Core

Two conventions govern this section. Every guarantee is about what a finite oracle certifies—G_{pool} where we can get it, G_k where the depth matters—and never silently about $G^\star$; and the guard is analyzed as an *index*, which changes the order in which stored counterexamples are consulted but not the worst case, so we claim a measured cost reduction and not an asymptotic separation. Statements and proofs of everything below are in the supplementary material.

Assumption 1 (Refutation soundness)**.** When $\mathcal{O}_k$ refutes a candidate it returns a genuine witness: if $\mathcal{O}_k(p) = x$ then $\neg\varphi(p, x)$.

Assumption 1 constrains *refutation*, not acceptance. It is the only oracle property the guard needs; it holds for any oracle reporting a failing execution it actually ran, and for ours it also requires the reference to be right, which the corpus gate enforces (Section VI). It has one soft edge: a *timeout* verdict is a resource fact rather than a witness of $\neg\varphi$, and the one fault whose candidates sit at the sandbox limit produces every cross-arm inconsistency we observe (Section VII-F). The converse—that acceptance implies correctness—is assumed nowhere: at depth k it is false in general, and Section VII-D measures how false.

Theorem 2 (Guard soundness)**.** *Under Assumption 1, if* $\mathrm{Guard}_{\mathcal{M}}(p) = 1$ *with witness* x *then* $p \notin G_{\mathrm{pool}}$*, and hence* $p \notin G^\star$*; and* $p \notin G_k$ *for every check set containing* x*, which includes every* X_k *that exhausts the pool. No candidate such a call would have accepted is ever blocked, however widely the guard searches. If* CEGMEM *returns* $\hat{p}$*, then* $\hat{p}$ *passed every input its accepting call drew.*

The theorem is about the guard, not the oracle: Eq. (3) blocks only on a counterexample that is *re-executed and still fails*, so widening the search can only find more true refutations, and nothing here says $\hat{p} \in G^\star$. Its second clause can fail only on a redrawn X_k that omits the witness, so we test rather than assume: an audit condition pays the oracle on blocked rounds and asks whether it accepts, and over 4,411 blocked rounds it never did. That is not a tight bound on the hazard—where X_k exhausts the pool the audit confirms what already holds by

construction, and below $k = 100$, where the hazard is real, it was not run—but it does establish that the implemented guard blocks nothing the implemented oracle accepts. This is falsifier F1, with the requirement that a guarded arm's repair rate equal its unguarded twin's.

Corollary 3 (Budgeted success)**.** *For any guard sound in the sense of Theorem 2: if blocked rounds are* not *charged against the proposal budget B, the probability of repairing within B charged proposals cannot fall, and rises strictly only on realizations where a block moves the first accepting draw from beyond B to within it. If blocked rounds* are *charged, the guard is outcome-neutral: under common random numbers the round of first acceptance is invariant to it.*

Corollary 3 shaped the study and applies to untyped memory as much as typed. Under the accounting Section VII uses—one proposal, one budget unit—the guard cannot change which round first accepts, so any gap between a guarded arm and its unguarded twin is a soundness bug, not a result (Section VII-F); it also makes the first half untestable there, hence the free-guarded condition (Section VII-C).

a) Three further results, stated in full in the supplement.: Under an i.i.d. proposer and *ideal typing*—which also requires refutation to be class-exclusive, an idealization our λ index does not meet—typed memory halts on an unbounded run within $K{+}1$ oracle calls *and* $K{+}1$ proposals, K being the reachable failure classes, while untyped memory attains the call bound and no proposal bound. Expected oracle calls fall from $1/\pi$ to $1 + D$ with either memory, $D = \sum_\tau q_\tau/(q_\tau + \pi)$. And counting redundancy against the *observed-type history*—the classes already refuted this episode whether or not the arm records them, which is what makes it measurable in a memory-free arm—redundant attempts *paid* fall to 0 with either memory, while those *generated* fall to 0 only with typed memory, which needs steering to work; Section VII-C reports that it does not. Of the three budget quantities only two are bounded: not *charged* proposals, which include blocked rounds. These are directional predictions, and Section VII-E reports which of the closed forms the data supports.

b) What the index costs.: Our guard consults the λ-matched bucket first and then the rest of memory (Section V). Both it and a flat scan short-circuit at the first entry that still refutes, and both consult all of memory when none does, so their worst cases coincide and no asymptotic separation exists. What differs is where in the order a refuting entry is found: the index saves to the extent that refuting entries concentrate in the bucket—35% of guard decisions in vivo, a rate the random-partition control matches (Section VII-B)—and pays $|\mathcal{M}[\lambda(p)]|$ extra consultations on a miss. On this corpus there is little order to exploit: memory holds a median of one entry when the guard fires, and 86% of blocks resolve on the first consultation. Separately, deduplicating stored counterexamples by input and memoizing verdicts within a round reduce *executions* at unchanged *consultations*—which is the signature Section VII-C finds, and the reason our claim there is a measured constant factor.

V. Implementation

CEGMem is 4,700 lines of Python. Three components matter, plus one design error whose correction is the difference between a null result and the paper's main finding.

a) Proposer and oracle.: One LLM call per round returns a whole rewritten program in a fenced block. The prompt carries the buggy source, worked input/output examples (identical in every arm, so the specification cannot confound the comparison) and—only in the steered arm—the evidence and exclusion blocks of Eq. (4). Output budget is sized per program so a truncated reply cannot enter memory as a *harness* failure; truncations, overflows and backend errors are logged as spent rounds that leave memory untouched. The oracle $\mathcal{O}_k$ is a differential tester: candidate and reference run on a per-call sample X_k of the fault's shipped pool in a sandbox with a 30 s per-case limit, and the first disagreement is returned as a case *name*, so re-executing it later is cheap. Acceptance means agreement on every case in X_k; the reference is never visible to the proposer.

b) The type map, and the key the guard can use.: θ is the pair (edit location, violated property). The location is a pure diff against the buggy source and so is computable *before* execution, which is what makes it usable as the guard's index λ; the property is read off the failing execution—wrong output, uncaught exception, or timeout. We report the *fine* granularity (line-range location); a *coarse* variant collapsing it to the whole program measures how much the location half contributes (Section VII-B).

c) An implementation correction.: Our first guard searched the λ-matched bucket *and nothing else*, which makes the index a *partition*—sound as a stopping rule only if its key decides refutation. It does not: the key is where a candidate edits, refutation is decided by which input it gets wrong, and the two are close to independent. The frozen log prices the mistake exactly: of the typed rounds that reached the oracle and were refuted, 68.9% came back carrying a counterexample *already in memory*—every one a round the flat scan would have blocked. The fix is fifteen lines: search the bucket first, then fall back to the rest of memory, deduplicating by counterexample input and memoizing verdicts within the call—which is also why this paper claims no asymptotic advantage for the index, since the two guards then share a worst case. Every number here is from the corrected guard; we report the error because a bucket-only guard is the natural first implementation and it silently turns this paper's result into a null one.

d) Instrumentation and replay.: One record per round carries the patch, verdict, counterexample, both type granularities, the guard's accounting (entries consulted, re-executions, whether the bucket was hit and whether its counterexample fired), token counts, and wall-clock split between model, oracle sandbox and guard sandbox. Episode summaries come from

one function shared by every consumer, so "oracle calls to repair" cannot mean two things in two tables. Every model call is cached under a key containing the full prompt, model, temperature, output budget and the draw nonce $(\textit{task}, \textit{seed}, \textit{round})$, which omits the arm and so implements common random numbers (Section VI); a cell therefore replays byte-for-byte and a stale entry cannot answer a different prompt. Episode identity is a deterministic hash of the experiment cell, so an interrupted run rewrites its own rounds instead of appending a truncated duplicate.

VI. Study Design

We ask five questions: whether memory reduces the cost of repair and how that varies with difficulty (RQ1); whether *typing* the memory beats a flat counterexample log, and whether any advantage is attributable to the types being right rather than to partitioning at all (RQ2); which operation does the work, guarding or steering (RQ3); whether an accepted patch is a repaired program and how that depends on oracle depth (RQ4); and whether the theory's assumptions hold on real faults, including what typed steering costs when it misfires (RQ5).

A. Corpus and protocol

We use ConDefects [25], whose faults are AtCoder contest submissions from 2021–2023, curated to limit the training-data leakage that afflicts older benchmarks. Corpus construction is frozen and takes 526 examined candidates to 99: -64 whose reference disagreed with the shipped output, -48 too slow, -7 unresolved, -112 because a coding task may contribute one fault, and -196 never screened at all (the supplement). That last figure describes a sequential procedure honestly: candidates were screened at 40 model calls each and admitted until each band's quota filled, so this is a quota sample, not a probability sample of a characterized pool. The 99 are a strict subset of a previously frozen 106, so nothing entered on the basis of results. The *oracle gate* admits a fault only if the reference reproduces the shipped outputs, at least one shipped case exposes the fault, and its natural mutants are caught—which is what makes Assumption 1 hold by construction. The *slow-reference filter* then drops a fault whose reference needs over 10 s on one of its first 20 cases. Its unit is the wrong one—cost is per *call*—and 45 of the 99 faults it kept still hold a call over that same threshold, the slowest 504 s. What it removed is *difficulty* (2042 against 940), a correlate of oracle cost rather than oracle cost; Section IX measures which way that biases us.

The proposer is `qwen2.5-coder:7b` (Q4_K_M, served locally) at temperature 1.0, budget $B = 20$ proposals, oracle depth $k = 100$, 30 s sandbox timeout; the supplement fixes the rest. Temperature is deliberately not lowered for determinism: the loop rests on redraw being meaningful, and at low temperature the same prompt returns approximately the same patch, so no-memory would loop forever on one wrong patch and redraw-after-block would be vacuous—every arm would degenerate rather than the comparison becoming cleaner. The cost is that the steering result is scoped to $T = 1.0$ (Section IX).

Tasks are banded by the proposer's own per-round success rate $\hat{\pi}$, estimated from the no-memory arm run past its first accept so early stopping does not bias it: *dead* ($\hat{\pi} < 0.02$; 23 tasks), *hard* (17), *medium* (13), *easy* (25), *too-easy* (21), with pooled $\hat{\pi} = 0.207$ over 9,896 scored rounds. This is a *post hoc* covariate and not the design: the 40-call selection screen quantizes $\hat{\pi}$ to 0.025, so the *hard* band admits three attainable values and one success in forty separates it from *dead*. 42 of the 99 tasks report in a different band than they were selected into—*hard* $28 \to 17$, draining into *dead* $16 \to 23$—so the quota did not survive measurement, and per-band comparisons against no memory are inflated by regression to the mean. We report those columns as descriptive only; the typed-versus-untyped contrast within a band is unaffected, since under common random numbers the band comes from a draw the two arms share symmetrically.

B. Arms and pairing

Five arms switch the two memory operations independently: NO-MEM (both off, run past first accept for an unbiased $\hat{\pi}$), UNTYPED (unindexed guard, no prompt blocks), TYPED (indexed guard + steering), GUARD-ONLY and STEER-ONLY (one operation each). Four sweeps follow: oracle depth $k \in \{100, 20, 8, 3\}$; typing coherence $c \in \{1.0, 0.9, 0.75, 0.5, 0.25, 0.0\}$ plus a random-partition control; a redundancy audit that pays the oracle on blocked rounds to recover their classes; and a free-guarded-rounds condition (Corollary 3).

The draw nonce is $(\textit{task}, \textit{seed}, \textit{round})$ and *omits the arm*. Two arms whose prompt is byte-identical at a given round therefore receive the same completion, which pairs them on common random numbers [20] rather than buying the same draw repeatedly, and makes round 1 identical in every arm since memory is empty there. When prompts diverge, the prompt separates the cache keys, so nothing is shared that should not be. No sampling seed reaches the backend—the seed enters only the cache key—so a task's five seeds are five *independent* draws at $T = 1.0$: round 1 shares their prompt yet returns 4.43 distinct patches of five on average, never one. Pairing is memoization, not a shared RNG, and it has a consequence we state plainly: the draw table is fixed once drawn, so all inference is *conditional* on it and the replication axis is tasks $\times$ seeds, not re-sampling of the model.

C. Metrics and statistics

Two outcomes on the three main arms are **confirmatory**: repair rate at a matched oracle-call budget, and oracle calls per episode; everything else is exploratory. The primary metric is the fraction of cells repaired within b oracle calls, $b = 1 \ldots 20$, pre-specified rather than total calls because an arm with no guard loses a total-call comparison *by construction*. On the budget curve nothing is guaranteed: spending fewer calls helps only if the calls spent still find repairs.

Secondary metrics are oracle calls; *total program executions and the seconds they take*—every run of a candidate against a case, whether the oracle asked for it or the guard did; *paid* and

caught redundant verifications; entries consulted; tokens; and wall-clock. Redundancy is counted the same way in every arm: a blocked round carries no class of its own, so we recover it through the pairing from the byte-identical no-memory round the oracle already paid for. Where that recovery is unavailable—the steered arm, whose draws diverge—we report the censored fraction (52%).

Cells are paired on $(task, seed)$; the default test is the exact two-sided sign test with $\hat{A}_{12}$ [28] for effect size, and because seeds of one task share a fault, every headline comparison is also reported per task with a task-clustered bootstrap interval over 10,000 resamples (Table II). Sweep p-values are Benjamini–Hochberg adjusted [29]; audits observing zero failures get an exact one-sided Clopper–Pearson upper bound rather than a zero rate. Three falsifiers were pre-specified: a guarded arm must neither block a candidate the oracle accepts (Theorem 2) nor differ in repair rate from its unguarded twin (Corollary 3); redundancy *present* must be invariant across unconditioned arms, since they draw identically; and the random-partition control must *not* reproduce typing's advantage.

VII. Results

All numbers come from the frozen log via the extractors in the artifact (Section X); p-values are exact two-sided sign tests. Five seeds of one task share a fault, a model and a cache, so the cell is not an independent replication: Table II carries every headline comparison at both units with a task-clustered bootstrap interval, and the task-level result carries the claim. The confirmatory family is repair at a matched oracle budget and oracle calls, on the three main arms; everything else is exploratory.

A. RQ1: what memory buys

Over all 495 paired cells (Table II), memory cuts oracle calls per episode from 9.47 to 1.91 (untyped) and 1.83 (typed)—$\times$4.95 and $\times$5.17—and *paid* redundant verifications, where a proposal whose class was already refuted still reaches the oracle, from 4.48 to 0.101 and 0.079. Repair rate is unchanged (0.703, 0.699, 0.697; flips 36/37, $p = 1.0$), so this is not accuracy traded for cost.

a) The guard executes, and the accounting says so.: A guard that declines a validation still runs a stored counterexample, so a comparison denominated only in oracle calls would hide its cost. Ours does not: the execution counter bills every guard replay in every arm. Counting that way, total executions fall from 95.68 to 43.01, $\times$2.22, on 251 of 364 discordant cells and 63 of 93 discordant tasks ($p = 0.0008$), with the guard's share at 22%; in seconds the same comparison is 72.9 $\rightarrow$ 38.2, so the reduction is not an artifact of the unit. The split is conservative against the memory arms: a consultation is billed as an execution even when deduplication or memoization means no process started. Per round the trade is legible, and it is an ordering trade (Section II-B): in TYPED a round the guard resolves costs 1.13 executions against 18.9 for one that reaches the oracle. Those surviving calls are dearer than

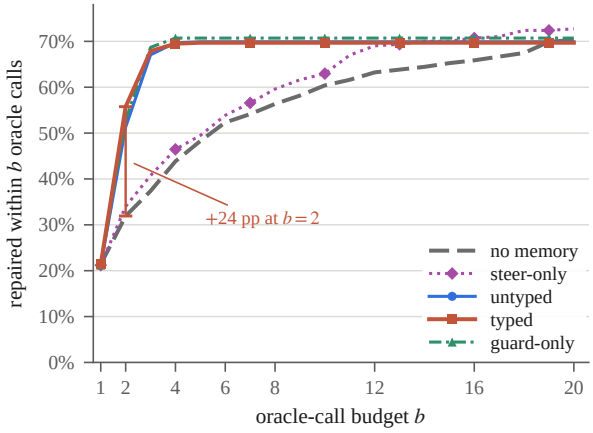

Fig. 1. The pre-specified primary metric: repair rate at a matched *oracle-call* budget b, five arms, 495 main and 297 ablation cells. Memory saturates by $b = 4$; no memory needs $b = 20$ for the same rate. STEER-ONLY tracks no memory, which is Section VII-C's finding in one line.

no memory's (12.5) because the cheap refutations are what the guard reaches first—a selection effect that flatters neither arm, since both totals are reported. Typing does *not* reduce total executions against a flat log (44.45 vs. 43.01; 170/146, $p = 0.20$); it reduces the time those consultations take. The supplement gives both units per arm.

b) The saving is not spread evenly.: One oracle call costs between 0.06 s and 110 s depending on the fault, and the reduction tracks that. In terciles of oracle expense, sandbox seconds fall $\times$1.00, $\times$1.15 and $\times$2.07: where validation is cheap the guard's replays cost about what they displace, so no time is saved at all. Executions fall in every tercile ($\times$1.68 to $\times$2.66) because that unit does not price a case (the supplement). The mechanism buys *time* only where validation is expensive, which is also what makes the slow-reference filter's bias measurable (Section IX).

c) The primary metric.: Figure 1 gives repair rate against a matched oracle-call budget. The arms coincide at $b = 1$ up to the timeout-edge cell. At $b = 2$ they separate—31.9% (no memory) against 51.3% (untyped) and 55.8% (typed)—and the memory arms are within 0.3 pp of their asymptote by $b = 4$, where no memory has reached 43.8%. No memory needs the full $b = 20$ to reach 70.3%; GUARD-ONLY reaches 70.7% at $b = 4$. The gain survives the conservative unit: the task-clustered interval on the paired difference at $b = 2$ is $[18.4, 29.5]$ pp.

Counted in *proposals* rather than oracle calls the arms nearly coincide (0.564 / 0.560 / 0.578 at eight charged proposals), exactly as Corollary 3 predicts when blocked rounds are charged: memory converts the expensive resource into repairs, it does not make the model smarter.

RQ1. Memory converts verification budget into repairs: $\times$5.2 fewer oracle calls, $\times$2.2 fewer program executions counting guard replays, and +23.8 pp repair at a two-call budget, at unchanged repair rate.

TABLE II

MAIN COMPARISON, ALL 495 PAIRED CELLS (99 FAULTS × 5 SEEDS), MEANS PER EPISODE. GUARD REPLAYS ARE BILLED AS PROGRAM EXECUTIONS. BECAUSE SEEDS OF ONE TASK SHARE A FAULT, A MODEL AND A CACHE, THE PER-CELL SIGN TEST IS ANTI-CONSERVATIVE, SO EACH ROW ALSO CARRIES THE PER-TASK TEST AND A TASK-CLUSTERED BOOTSTRAP INTERVAL ON THE PAIRED DIFFERENCE (10,000 RESAMPLES OF THE 99 TASKS); THE TASK-LEVEL RESULT CARRIES EACH CLAIM, AND EVERY COMPARISON KEEPS ITS SIGN AND SIGNIFICANCE THERE. ROWS WHERE TYPED IS *worse* ARE MARKED, BECAUSE TWO OF THEM ARE FINDINGS.

	NO-MEM	UNTYPED	TYPED	**per cell, ty vs. un**	**per task**	**clustered CI**
Repair @ B=20	0.703	0.699	0.697	$36/37$, $p=1.0$	parity	—
Repair @ b=2 calls	0.319	0.513	**0.558**	primary metric; gain over NO-MEM		$[+0.184, +0.295]$
Oracle calls / ep.	9.47	1.91	**1.83**	$93/59$, $p=0.007$	$36/19$, $p=0.030$	$[-0.141, -0.018]$
vs. NO-MEM		×4.95, ×5.17		$325/10$, $p<10^{-80}$	$88/0$, $p<10^{-26}$	$[-9.07, -6.27]$
Program exec. / ep.	95.68	44.45	**43.01**	$170/146$, $p=0.20$	vs. NO-MEM:	
vs. NO-MEM		×2.22		$251/113$, $p<10^{-12}$	$63/30$, $p=0.0008$	$[-84.1, -28.3]$
Redundant *paid* / ep.	4.481	0.101	**0.079**	vs. NO-MEM: ×44.4, ×56.9		$[-5.37, -3.52]$
Entries consulted / ep.	0.00	10.20	9.38	$163/131$, $p=0.07$		
Prompt tokens / ep.	**4,546**	4,592	6,727	$+\mathbf{46}\%$, $p<10^{-19}$		
Wall-clock s / ep.	108.8	**77.6**	140.5	$+\mathbf{81}\%$, $p=0.043$		

B. RQ2: does typing beat a flat log?

Typed uses fewer oracle calls than untyped on 93 of 152 discordant cells ($p=0.007$, $\hat{A}_{12}=0.524$) and on 36 of 55 discordant tasks ($p=0.030$; signed-rank $p=0.023$), with a clustered interval on the paired difference of $[-0.141, -0.018]$ calls. Pooled over the three primary bands the margin is $71/39$ ($p=0.0029$). The task-level version is the claim: real, signed, and small—1.83 against 1.91 calls.

One caveat: TYPED differs from UNTYPED in two ways at once—indexing the guard by λ *and* putting exclusion blocks in the prompt—so the difference above is the package's, and Section VII-C separates them.

a) Is it the types, or just partitioning?: Any typed index invites the objection that *any* partition would do, so the pre-specified control files each refutation under a location drawn uniformly from those seen so far—memory still partitions, but the partition carries no information about failure class. On the same 90 paired sweep cells exactly one quantity responds: the redundant proposals the guard *recognizes*, 4.80 at $c=1.0$ against 4.47 for the control and 4.04 at $c=0$, correct typing winning $9/9$ ($p=0.004$). Nothing else moves—paid redundancy ties on all 90; guard seconds, entries consulted, oracle calls and repair rate all give $p>0.6$; and the bucket-only resolution rate is flat in c (33.1–35.3%) with the control inside it, because an uninformative partition still *fills* buckets.

RQ2. The licensed claim is narrower than "typing helps": typing determines *which* redundant proposals the guard recognizes, does not affect blocking correctness (guaranteed by the fallback scan, Theorem 2), and on this sweep does not measurably affect guard time.

C. RQ3: which operation does the work?

a) The guard: identical outcomes, cheaper answers.: GUARD-ONLY and UNTYPED share their draws, so they should produce identical episodes and differ only in bookkeeping—and they do, on 295 of 297 cells, the two exceptions being the timeout-edge task (Section VII-F). The indexed arm repairs 0.707 against 0.704 and consults 9.74 against 10.02 ($19/13$, $p=0.38$), while guard seconds fall $6.66 \to 4.04$ on 191 of 231 episodes ($p<10^{-24}$). The *sign* is solid, the *magnitude* is not: the task-clustered interval is $[-6.5, -0.5]$ s and one fault supplies 1.8 s of the baseline (the supplement). Holding the episode fixed isolates the index-cost proposition—the same work, answered faster—but the 35% of decisions the λ bucket settles is a rate the random-partition control also reaches, so the clock records deduplication, not the index.

b) Steering: null on every outcome, and expensive.: STEER-ONLY keeps the prompt blocks and drops the guard. Round 1 matches no memory on all 297 cells and trajectories diverge where the first evidence block enters; after that nothing moves: repair $0.707 \to 0.727$ ($23/17$, $p=0.43$), oracle calls $9.45 \to 8.81$ ($80/71$, $p=0.52$), paid redundancy $4.43 \to 4.16$. What moves is the bill: prompt tokens $4519 \to 10{,}296$, $+128\%$ ($64/170$, $p<10^{-11}$), growing $+5.9$ per round against untyped's -1.2.

The mechanism-level null is sharper than the outcome-level one. Length-matched by failed-round position, the rate at which the proposer re-draws an *already-eliminated* class is 50.2% without the exclusion block and 50.7% with it. The model conditions on the block—trajectories shift immediately—but does not avoid the classes it names. Equation (4)'s renormalization is requested and not granted.

The consequence is an ordering on the clock. Over the 297 shared cells: GUARD-ONLY 68.9 s per episode, UNTYPED 80.4, STEER-ONLY 98.2, NO-MEM 110.8, TYPED 155.0. On a local 7B prompt length is latency, so typed's prompt tokens become $+93\%$ wall-clock over untyped on these cells, and $+81\%$ on the main grid, where the paired test is 224 wins to 270 ($p=0.043$). The arm that dominates every other on wall-clock, oracle calls and tokens at once is GUARD-ONLY—the index without the prompt blocks.

c) Does difficulty change the answer?: The band-wise numbers suggest it might: in the *dead* band—per-round success under 2%—repair rises from 2.6% to 12.2% (typed). So we

ran the interaction test, at the task unit, permuting band labels over tasks (20,000 draws), and it says two things the band-wise numbers did not. The full agent's effect *does* depend on difficulty ($p = 0.023$ across the five bands), but not as a dead-band bonus: *dead* gains +9.6 pp (CI $[2.6, 18.3]$) while *hard* loses -11.8 pp (CI $[-28.2, 4.7]$). And isolating the prompt half erases the pattern—STEER-ONLY against no memory gives a null interaction ($p = 0.87$) and a dead-band interval covering zero. The gain belongs to the whole agent, not to steering, and we withdraw the reading that it was evidence for the prompt half.

d) Free guarded rounds.: Under the reported accounting a blocked round costs a unit of budget, so Corollary 3 makes the guard outcome-neutral and its first half untestable. Charging a blocked round a model call but no budget unit, both memory arms gain repair at unchanged oracle cost (untyped $0.772 \to 0.860$, typed $0.815 \to 0.889$) while the guardless control is inert ($0.800 \to 0.783$). But the $9/0$ flips come from only 19 tasks, and permuting the free/base label a task at a time gives $p = 0.061$: suggestive, not confirmed (the supplement).

RQ3. The guard carries the value—identical episodes, cheaper guard time, 14% less wall-clock than untyped, no token overhead—while prompt-side steering moves no outcome metric at $+128\%$ prompt tokens and no band survives its interaction test. GUARD-ONLY is the configuration to deploy.

D. RQ4: is an accepted patch a repair?

Every accepted patch is re-scored against the whole shipped pool. Because X_k exhausts that pool for 97 of the 99 faults at $k = 100$ (Section III), acceptance there usually already *is* pool adequacy: all 1,256 audited acceptances pass with 0 overfitting, and 0 of the 939 carrying a regression verdict break a case the buggy version passed—the denominators differ because a regression verdict needs at least one such case and 317 acceptances belong to faults with none. A planted-mutant study finds $249/249$ non-equivalent mutants caught (48 of 297 plants were equivalent). Three zero counts are not three zero rates: the one-sided 95% upper limits are 0.24%, 0.32% and 1.2%. What this establishes is membership in G_{pool}, not $G^\star$.

Sweeping the depth shows why that matters. Raw acceptance *rises* as the oracle gets shallower, 0.678 at $k = 100$ to 0.867 at $k = 3$, and the audit says why: overfitting climbs 0%, 7.6%, 20.8%, 39.7% as the depth falls $100, 20, 8, 3$. Full-pool repair is monotone, with *no difference* between the two deepest (0.678 at both 20 and 100, at 55% of the executions) while $k \leq 8$ is unsafe (0.633, then 0.522). Below $k = 100$, X_k is a proper subset of most pools, and that is also where Theorem 2's second clause is unaudited.

RQ4. At the reported depth acceptance means pool adequacy; a five-fold shallower oracle preserves it, and at $k = 3$ two of five acceptances are false.

E. RQ5: assumptions, and what steering costs when it misfires

a) Informativeness measured, coherence not.: An offline study re-executes stored counterexamples against other candidates of the same label: the cross-refutation rate is 0.908 (95% CI $[0.884, 0.929]$) at fine granularity and 0.822 $[0.790, 0.852]$ at coarse, over 26,048 executions. This estimates ρ—how far a counterexample generalizes—which is what the guard's usefulness depends on. It is *not* an estimate of c, the rate at which memory attributes a candidate to its true class; that needs a ground-truth labelling the corpus does not supply, so we perturb c directly instead (Section VII-B). Earlier drafts called this a coherence check; it is not, and only ρ is validated here.

b) The theory is directional, not calibrated.: Fitting the redundancy theorem's no-memory form across 79 tasks gives $r = 0.657$ with relative MAE 1.33, an order-of-magnitude prediction the data respects in shape; the memory-arm form could not be fit at all, its predicted spread being smaller than the sampling noise. Their i.i.d. assumption is what a real proposer violates, and the violation is the redundancy a guard exists to intercept (Section II-C).

c) Anchoring: the cost of excluding a class.: Typed steering can exclude the class the *correct* fix belongs to. Over 1,503 steered episodes at $c = 1.0$ the correct patch's edit location had entered the exclusion block in 5.7% of episodes that then failed, and the rate is highest in exactly the band where steering helps most (19.7% in *dead*, 0 in the two easiest). Steering is a high-variance intervention worth making only where the baseline is near zero.

F. Integrity of the comparison

Common random numbers hold exactly: round 1 is byte-identical to the no-memory arm on $495/495$ main-grid and $297/297$ ablation cells in all four treatment arms, the frozen grid has 0 missing cells of 1,485, and the three pre-specified falsifiers behave. A single fault whose candidates run at the 30 s sandbox edge produces every cross-arm inconsistency in the study, including the two non-identical guard-only episodes. We left the frozen timeout unchanged and repeat the headline numbers without it in the supplement.

VIII. Discussion

The two halves of typed memory fail and succeed for the same reason. Obeying Eq. (4) would need the proposer to simulate the oracle's verdict on a class described to it only in the abstract, which is harder than writing a patch; the guard needs no such simulation, since it re-executes a concrete counterexample and reads the answer. Hence the model *conditions* on the exclusion block—trajectories diverge the moment it appears—while re-drawing the excluded classes at an unchanged rate, and hence the concurrent text-to-SQL work [22] reports the same non-separation. A class label is worth more as an *index over evidence* than as an *instruction to a model*; the design consequence is to count oracle calls and executions rather than tokens, since the resource identified as squandered [7] is invisible in a token budget.

IX. Threats to Validity

a) Construct validity.: Our primary metric counts *oracle calls*, the resource recent measurement identifies as squandered [7], [8]. It is not the only cost model—an agent billed per token, or one whose suite is cheap, would rank the arms differently, and our own wall-clock row shows the full typed agent losing under a latency-weighted one—so we report oracle calls, program executions, tokens and wall-clock separately and name the resource each claim is about. Guard replays are billed as executions and we claim no avoided execution: a full X_k validation is replaced by one that terminates on its first case, an ordering effect (Section II-B) that survives the accounting. And "repair" is acceptance, a claim about G_k rather than a fact about $G^\star$; we audit it against G_{pool} with exact bounds, and no test-based procedure can certify $G^\star$.

b) Internal validity.: Common random numbers make the comparison tight but make all inference *conditional on a fixed draw table*: the replication axis is tasks $\times$ seeds, not re-sampling of the model. Difficulty bands come from the no-memory arm's own $\hat{\pi}$, so per-band comparisons against no memory are inflated by regression to the mean and are descriptive only. The timeout-edge fault of Section VII-F accounts for every cross-arm inconsistency; removing it leaves the cost and repair results unmoved but cuts the guard-second reduction from -39% to -18% (the supplement), so that magnitude rests on one fault and we do not lean on it. Typed's *caught*-redundancy count is 52% censored, because a blocked round in the steered arm has no paired no-memory draw to recover its class from; the claim in Table II rests on *paid* redundancy, uncensored in every arm.

c) External validity.: Scope is the strongest limitation. *One model, one temperature*: every number is `qwen2.5-coder:7b` at $T = 1.0$, and the steering null may be scale-dependent, since a larger model might obey an instruction a 7B ignores; we predict, and do not test, that the guard result transfers, it being a property of the runtime. *One fault distribution, and a filtered one*: ConDefects faults are single-file competitive-programming submissions with a rich shipped pool, which is what makes a clean counterexample oracle possible and is unlike a repository-scale issue. The slow-reference filter thinned the harder faults; rather than assert which way that biases us we measure it inside the kept corpus, where oracle cost still spans a factor of 1,800: the saving in sandbox seconds is $\times 1.00$ in the cheapest third of faults and $\times 2.07$ in the dearest (Section VII-A), so the aggregate sits in exactly the regime the filter thinned, and the dropped faults would have deepened it rather than reversed it. Because the corpus is a quota sample (Section VI), none of this is a population estimate; the paired design is what carries the claims. *One scale of memory*: episodes run at most 20 rounds and memory holds a median of one entry when the guard fires, which is why we claim nothing for the index.

d) Statistical validity, and baselines we did not run.: A per-cell test overstates precision, so every headline comparison is reported at both units with a task-clustered interval (Table II); all keep their sign there, with typed-versus-untyped weakening from $p = 0.007$ to $p = 0.030$ as it should. Only two outcomes on three arms are confirmatory. The dead-band result now has the interaction test it lacked (Section VII-C), and the test does not support the reading we first gave it. What survives is a difficulty dependence of the *whole* agent, one band up and one down on 23 and 17 tasks, which we report as exploratory. Finally, the comparison a reader will want is against a per-modification-point prioritizer [16], the mechanism our guard is closest to. We have not implemented or run that arm, so Table I is argued mechanically rather than measured: the boundary it draws is a claim about mechanism, not a measured difference.

e) Reproducibility, and what "pre-specified" means.: We say *pre-specified* rather than pre-registered: metrics, arms, falsifiers and the banding rule were fixed in a design document and in the frozen corpus and protocol artifacts before the reported run, timestamped in the archive, but there is no third-party registry entry. One chronology bears on that: the guard correction of Section V was made after inspecting an *earlier* run's accounting and *before* the reported run.

X. Conclusion

LLM repair agents run tests indiscriminately and most of that execution does not pay [7], [8]. Classical repair cuts that work either by relating candidate patches to one another [13], [14], which an unenumerable candidate space rules out, or by ordering the evidence so a doomed candidate dies on the first case [15], [16]. This paper carries the second mechanism into an LLM loop and changes what it orders from: counterexamples retained across candidates, where an input already in memory *proves* that a candidate reproducing it will fail. Memory cuts oracle calls $\times 5.2$ at unchanged repair rate and, counting every guard replay as the execution it is, total program executions $\times 2.2$. That there is anything to intercept is itself a finding: one proposal in four is byte-identical to an earlier one, a rate that is zero by construction in the synthesis loop the design came from.

Two results change the recommendation. Steering does nothing at this scale, at $+128\%$ prompt tokens, and makes the full agent the slowest of five arms; and the λ *ordering* is indistinguishable from a flat scan on memories this small—the typed guard's 39% cheaper seconds come from deduplication, not the index. What we recommend is therefore not the architecture we set out to validate but the guard alone. Whether it survives a per-modification-point prioritizer (Section II-B), another model scale, or the expensive-oracle faults we filtered out, are the three questions we would want answered next.

Data Availability

One archive holds the source, the frozen episode log (39,147 rounds), the corpus and banding artifacts, the prompt templates, a pinned model manifest and lockfile, the response cache, and the six extractors that regenerate every number here from it without a GPU. *Access:* `[anonymized DOI]`.

REFERENCES

[1] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 2188–2200.

[2] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 1592–1604.

[3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, pp. 50 528–50 652.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025, the arXiv version is titled "Agentless: Demystifying LLM-based Software Engineering Agents" (arXiv:2407.01489).

[5] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 819–831.

[6] A. Solar-Lezama, L. Tancau, R. Bodík, S. A. Seshia, and V. Saraswat, "Combinatorial sketching for finite programs," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2006, pp. 404–415, introduces the counterexample-guided synthesis loop; the term "CEGIS" does not appear in the paper.

[7] Z. Lin, J. Zhu, M. Zhou, X. Wang, Z. Sun, R. Yang, D. Lo, and L. Li, "To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair," in *Proceedings of the 35th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2026, to appear.

[8] Z. Chen, Z. Sun, Y. Shi, C. Peng, X. Gu, D. Lo, and L. Jiang, "Rethinking the value of agent-generated tests for LLM-based software engineering agents," *arXiv preprint arXiv:2602.07900*, 2026.

[9] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[10] L. Chen, Y. Ouyang, and L. Zhang, "Fast and precise on-the-fly patch validation for all," in *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2021, pp. 1123–1134.

[11] Y.-A. Xiao, C. Yang, B. Wang, and Y. Xiong, "Accelerating patch validation for program repair with interception-based execution scheduling," *IEEE Transactions on Software Engineering*, vol. 50, no. 3, pp. 618–635, 2024.

[12] R. Ehrlich, B. Brown, J. Juravsky, R. Clark, C. Ré, and A. Mirhoseini, "CodeMonkeys: Scaling test-time compute for software engineering," *arXiv preprint arXiv:2501.14723*, 2025.

[13] W. Weimer, Z. P. Fry, and S. Forrest, "Leveraging program equivalence for adaptive program repair: Models and first results," in *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2013, pp. 356–366.

[14] S. Mechtaev, X. Gao, S. H. Tan, and A. Roychoudhury, "Test-equivalence analysis for automatic patch generation," *ACM Transactions on Software Engineering and Methodology*, vol. 27, no. 4, pp. 1–37, 2018.

[15] Y. Qi, X. Mao, and Y. Lei, "Efficient automated program repair through fault-recorded testing prioritization," in *2013 IEEE International Conference on Software Maintenance (ICSM)*, 2013, pp. 180–189.

[16] Y. Venugopal, P. Quang-Ngoc, and L. Eunseok, "Modification point aware test prioritization and sampling to improve patch validation in automatic program repair," *Applied Sciences*, vol. 10, no. 5, p. 1593, 2020.

[17] Y. Lou, J. Yang, S. Benton, D. Hao, L. Tan, Z. Chen, L. Zhang, and L. Zhang, "When automated program repair meets regression testing—an extensive study on two million patches," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 7, pp. 1–23, 2024.

[18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 8634–8652, author list follows the NeurIPS camera-ready.

[19] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[20] A. M. Law and W. D. Kelton, *Simulation Modeling and Analysis*, 3rd ed. Boston, MA: McGraw-Hill, 2000, common random numbers: Chapter 11, Section 11.2.

[21] F. Mu, J. Wang, L. Shi, S. Wang, S. Li, and Q. Wang, "ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair," in *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, 2026, to appear.

[22] K. N. H. Vo, T. M. Chu, A. T. D. Dinh, T. V. H. Bui, and T. Quan, "Causal episodic memory for feedback-driven agent repair," *arXiv preprint arXiv:2608.05906*, 2026.

[23] P. Orvalho, M. Janota, and V. M. Manquinho, "Counterexample guided program repair using zero-shot learning and maxsat-based fault localization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 649–657.

[24] Y. Xiong, X. Liu, M. Zeng, L. Zhang, and G. Huang, "Identifying patch correctness in test-based program repair," in *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM, 2018, pp. 789–799.

[25] Y. Wu, Z. Li, J. M. Zhang, and Y. Liu, "ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*. ACM, 2024, pp. 642–646.

[26] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2014, pp. 437–440.

[27] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[28] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[29] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995.